

(AMR)Warehouse Robot Navigation

A Comparative Study of BFS, Dijkstra and A for Pathfinding in a Warehouse*

Detail	Information
Core Algorithms	Breadth-First Search (BFS), Dijkstra's Algorithm, A* Search
Technologies	Python, Pygame, Streamlit
Submitted by	YASHVARDHAN SINGH GAUR (25MVCSMR0540) JAMPALA PURNASENA (25MVCSMR0491) BISWAJIT BEHERA (25MVCSMR0472) MANTHAN BIRADAR (25MVCSMR0512)
Guide / Faculty	DR.G KALPANA DR.K SRIKAR GOUD KADINTI GANGASHAR SAIBHARGAV KAMARAPU
Department & Institution	MALLA REDDY TECHNICAL CAMPUS
Date	03 October 2026

1. Abstract & Introduction

Abstract

Modern warehouses depend on robots that carry goods between shelves and pickup points. Each robot has to decide, again and again, which route to take. A poor choice wastes time and energy, and in a crowded warehouse it can mean collisions or blocked aisles.

In this project we built an interactive simulator where a robot navigates a warehouse grid filled with shelves and obstacles. We implemented three classic algorithms, BFS, Dijkstra and A*, and ran them on the same maps. We compared them on path length, path cost and the number of cells explored. The simulator also shows each algorithm searching step by step, so users can see why a particular path was chosen. In our experiments, A* found optimal paths while exploring far fewer cells than Dijkstra and BFS.

Introduction

Think of a warehouse as a city with narrow streets. Shelves are the buildings, and the robot is a delivery driver who has to reach a pickup point without driving through walls. Real warehouses go further: some aisles are one-way or crowded, and some turns are slower than straight runs.

Pathfinding is a classic topic in the Design and Analysis of Algorithms (DAA), and it is easy to learn the theory and still not feel the difference between algorithms. We wanted a project where you can watch them work. Our system turns a warehouse into a graph, lets three algorithms solve the same problem, and puts the results side by side.

2. Problem Statement, Objectives & Scope

Problem Statement

Given a warehouse layout containing shelves and obstacles, a start position for the robot and a pickup point, find the shortest path from start to goal. Then compare the shortest-path algorithms using the distance travelled and the number of cells explored.

Objectives

1. Model the warehouse as a graph that the algorithms can search.
2. Implement BFS, Dijkstra and A* and check that each returns a valid path.
3. Compare the algorithms on path length, total cost and explored cells.
4. Make the search visible, showing visited cells, the frontier and the final path, so users understand the result and not just a number.
5. Present the results clearly, through a visual simulator and a summary comparison.

Scope

In scope:

- A 2D grid warehouse with shelves and obstacles
- Adjustable start and goal positions and map layouts
- Uniform-cost and weighted-cost movement, such as congested aisles costing more
- Side-by-side comparison of the three algorithms

Out of scope:

- Multiple robots moving at the same time and collision negotiation between them
- Real hardware, sensors and physical robot dynamics
- Dynamic obstacles that move while the robot is travelling

3. Features & Functional Requirements

Key Features

- **Editable warehouse map:** place or remove shelves and obstacles, and move the start and pickup points.
- **Three algorithms in one place:** choose BFS, Dijkstra or A* and run them on the same map.
- **Step-by-step visualisation:** watch explored cells spread outwards and the final path being drawn.
- **Live metrics:** path length, total cost, explored cells and execution time shown after each run.
- **Comparison view:** all three algorithms' results side by side.
- **Weighted terrain (optional):** some cells cost more, which shows where BFS stops being reliable.

Functional Requirements

ID	Requirement
FR1	The system shall accept a grid layout with start, goal and obstacle cells.
FR2	The system shall represent the grid as a graph with valid neighbour connections.
FR3	The system shall compute a path using BFS, Dijkstra and A*.
FR4	The system shall report "no path found" when the goal is unreachable.
FR5	The system shall record the explored cells, path length and cost for each run.
FR6	The system shall visualise the search order and the final path.
FR7	The system shall display a comparison of the algorithms' results.

Non-Functional Requirements

- **Clarity:** anyone should be able to understand what the robot is doing.
- **Responsiveness:** typical grids should solve in a fraction of a second.
- **Ease of use:** no setup beyond running the app.

4. Algorithms & Methodology

Modelling the Problem

We represent the warehouse as a grid in which every walkable cell is a node. Each cell connects to its neighbours (up, down, left, right), and shelves and obstacles are simply left out of the graph. Every move has a cost, which is 1 by default and higher in congested areas.

Breadth-First Search (BFS)

BFS explores the map level by level, like ripples spreading from a stone dropped in water. It uses a **queue**: cells closest to the start are expanded first.

- It guarantees the fewest steps (minimum hops) when all moves cost the same.
- It does not account for varying costs.
- **Time complexity:** $O(V + E)$

Dijkstra's Algorithm

Dijkstra keeps a **tentative cost** for every cell. It repeatedly picks the unexplored cell with the lowest cost so far, then updates its neighbours' costs. Once the goal is reached, the path is rebuilt by following parent pointers backwards.

- It guarantees the least-cost path as long as costs are non-negative.
- It explores in all directions, even away from the goal.
- **Time complexity:** $O((V + E) \log V)$ with a priority queue (min-heap)

A* Search

A* is Dijkstra with a sense of direction. For each cell it computes

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost already travelled and $h(n)$ is a heuristic estimate of the cost remaining. We used the **Manhattan distance** as the heuristic, since the robot moves in four directions. It never overestimates the true cost, so A* stays optimal.

- It focuses the search toward the goal, so it usually explores far fewer cells.
- Its performance depends heavily on the quality of the heuristic.
- **Time complexity:** $O(E)$ in the best case and exponential in the worst case, though it is generally much faster than Dijkstra in practice.

Quick Comparison

Feature	BFS	Dijkstra	A*
Data structure	Queue	Priority queue	Priority queue
Handles weighted costs	No	Yes	Yes
Uses heuristic	No	No	Yes
Guarantees optimal path	Only for uniform costs	Yes	Yes (admissible heuristic)
Typical cells explored	High	High	Low

Methodology

1. Build the warehouse grid.
2. Convert it into a graph.
3. Run each algorithm on the identical map.
4. Record the path, cost, explored cells and time.
5. Visualise and compare.
6. Repeat on several layouts (open floor, dense shelves, maze-like aisles, blocked goal).

5. Technologies & Implementation

Technologies Used

Technology	Purpose
Python	Core language for the algorithms and logic
Pygame	Animated grid simulation of the robot's search
Streamlit	Web interface for controls, metrics and the comparison dashboard
heapq / collections.deque	Priority queue for Dijkstra and A*, queue for BFS

Implementation Overview

We split the project into separate parts so each one stays simple and testable:

- **Grid / Graph module:** builds the warehouse, validates cells and returns neighbours.
- **Algorithms module:** one function each for BFS, Dijkstra and A*, all returning the same result structure (path, cost, explored cells).
- **Visualisation module:** draws the grid, explored cells, the frontier and the final path.
- **Interface:** the Streamlit dashboard where the user picks the algorithm, edits the map and reads the results.

Keeping the return format identical for all three algorithms made the comparison simple. The same code can display any algorithm's output.

6. Results, Testing & Performance

Testing Approach

We tested on several layouts to check both correctness and behaviour:

Test Case	Purpose	Expected Outcome
Open floor, no obstacles	Baseline	All find an optimal path
Dense shelves	Obstacle handling	Valid path avoids all shelves
Maze-like aisles	Long, winding routes	Same optimal cost from Dijkstra and A*
Goal fully blocked	Edge case	"No path found" reported
Weighted congestion zones	Cost handling	Dijkstra and A* avoid costly cells, BFS may not
Start equals goal	Edge case	Zero-length path

What We Observed

These are the typical patterns for these algorithms. Please confirm them against your own numbers before submitting.

- On uniform-cost maps, all three found paths of the same length, as expected.
- Dijkstra and A* always matched in path cost, which confirms that the Manhattan heuristic is admissible.
- A* explored noticeably fewer cells than BFS and Dijkstra, especially when the goal was far from the start in open areas.
- On weighted maps, BFS gave the shortest path in steps but not the cheapest one, which shows why cost-aware algorithms matter.
- In very maze-like layouts, A*'s advantage shrank because walls force the search to explore widely anyway.

7. Conclusion, Future Scope & References

Conclusion

This project showed us that "shortest path" depends on what you measure and what you know about the map. BFS is simple and perfect when every step costs the same. Dijkstra handles real costs but searches blindly in every direction. A* brings in a sense of direction and reaches the same optimal answer with much less work. Building the visual simulator also helped us understand the algorithms better than the textbook pseudocode alone. For a warehouse robot, where speed and efficiency matter, A* is the most practical choice among the three.

Future Scope

- **Multi-robot coordination:** avoid collisions and deadlocks between several robots.
- **Dynamic obstacles:** replan on the fly with D* Lite or similar methods.
- **Diagonal and smoother movement:** use Euclidean or octile heuristics.
- **Multiple pickup points:** order the stops efficiently, a Travelling Salesman-style problem.
- **Real-world integration:** connect with real warehouse maps and robot hardware.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press.
- [2] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, 1959.
- [3] P. E. Hart, N. J. Nilsson, B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, 1968.
- [4] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
- [5] Python documentation: <https://docs.python.org/3/>
- [6] Pygame documentation: <https://www.pygame.org/docs/>
- [7] Streamlit documentation: <https://docs.streamlit.io/>